DSA

DATA STRUCTURES & ALGORITHMS

Combined Colourful Master Notes

Exam-focused • Beginner-friendly • Definitions • Algorithms • Dry runs • Complexity • Practice

Compiled from your four uploaded PDFs:
Unit I — Introduction to Algorithms & Analysis
Data Structures Introduction • Arrays & Lists • Stacks & Queues

How to use these notes

These notes combine overlapping material from all four uploaded PDFs into one revision resource. Core definitions and named topics follow the uploaded material; examples and study explanations are simplified for revision.

- **First pass:** learn the definition and the diagram/idea of each structure.
- **Second pass:** trace each algorithm with a small input by hand.
- **Exam pass:** write definition → working/principle → algorithm or diagram → example → complexity → applications.

Scope note: the uploaded Unit I algorithm PDF covers algorithm fundamentals, efficiency analysis, asymptotic notation, and recursive/non-recursive analysis. The other PDFs focus on data-structure basics, arrays/lists, stacks, queues, and related operations.

Contents

- 1. Data structures: fundamentals and classification
- 2. ADT and list ADT
- 3. Arrays and basic array operations
- 4. Linked lists: singly, circular and doubly linked
- 5. Stack: operations, representations and applications
- 6. Expressions: infix/postfix and postfix evaluation
- 7. Queue: operations, representations and queue types
- 8. Algorithms: definition, properties and specification
- 9. Algorithmic problem-solving process and problem types
- 10. Time/space complexity and analysis framework
- 11. Asymptotic notations: O, Ω and Θ
- 12. Non-recursive and recursive analysis
- 13. Complexity cheat sheet and common comparisons
- 14. Exam-ready long-answer frameworks
- 15. Practice questions with answer cues

1. Data structures: fundamentals and classification

What is a data structure?

A data structure is a way of organizing, storing, and retrieving data so that it can be processed efficiently. Choosing an appropriate structure affects the efficiency of operations such as searching, insertion, deletion, and traversal.

Basic terminology

Term	Meaning	Example
Data	Raw values or facts.	A student's name and ID.
Information	Processed/organized data with meaning.	A student's calculated grade.
Data item	One unit of data.	Age = 19.
Group item	A data item made of sub-items.	Name → first, middle, last.
Elementary item	An item not divided into smaller data items in the given context.	A single ID value.
Entity	An object with attributes/properties.	Student, employee.
Attribute	A property of an entity.	Name, age, course.
Field	A single elementary information unit representing an attribute.	Student ID field.
Record	A collection of related fields for one entity.	One student's complete row.
File	A collection of related records.	Student records file.
Key / primary key	A field whose value uniquely identifies a record.	Unique student ID.

Classification

Category	Meaning	Examples
Primitive	Fundamental language-supported types.	Integer, real/float, character, Boolean.
Non-primitive	Built from primitive types.	Array, list, stack, queue, tree, graph.
Linear	Elements are arranged in sequence.	Array, linked list, stack, queue.
Non-linear	Elements have hierarchical or network relationships.	Tree, graph.
Static	Size/memory capacity is fixed or allocated in advance.	Typical fixed-size array.
Dynamic	Memory/size can change as required.	Linked list.

Why do we need data structures?

- Efficiency: a good structure can reduce the time needed to find or update data.
- Organization: related values can be represented systematically.
- Reusability and abstraction: a structure can be reused behind a well-defined interface.
- Scalability: helps manage large collections and repeated operations.

Common operations

Operation	Meaning
Traversal	Visit/process each element.
Searching	Find an element or its location.
Insertion	Add a new element.
Deletion	Remove an element.
Sorting	Arrange elements in a chosen order.
Merging	Combine compatible lists/files.

Exam line: Algorithms + Data Structures = Programs. An algorithm describes the steps; a data structure organizes the data those steps operate on.

2. Abstract Data Type (ADT) and List ADT

Abstract Data Type

An ADT is a logical or mathematical description of data and the operations permitted on it, without specifying the implementation details. It describes **what** operations are available, not exactly **how** they are implemented.

- Encapsulation / information hiding keeps implementation details away from the user.
- Benefits: modularity, reuse, clearer code, and the ability to change implementation without changing client code.

List ADT

A list is an ordered collection of elements, often written A■, A■, …, A■. The element at position i is A■; its predecessor is A■■■ and successor is A■■■ when those positions exist.

List operation	Purpose
Insert(x, i)	Insert x at a specified position.
Delete(x) / Delete(i)	Remove an element or the element at a position.
Find(x)	Return the location of x, if present.
Next(i)	Return the successor position.
Previous(i)	Return the predecessor position.
Print / Traverse	Display or visit all elements.
MakeEmpty	Remove all elements / reset the list.

Array-based list vs linked-list-based list

Feature	Array-based list	Linked list
Memory	Contiguous locations; capacity often fixed in basic implementation.	Nodes may be in non-contiguous locations.
Access by index	Direct, typically O(1).	Sequential traversal, typically O(n).
Insert/delete in middle	May require shifting O(n) elements.	Can be O(1) once the relevant node/position is reached; finding it may take O(n).
Extra memory	Little per-element overhead.	Pointer/link field(s) in each node.
Best fit	Frequent indexed access.	Frequent local insertions/deletions; size changes.

Do not confuse ADT with implementation: “List ADT” is the interface/behaviour; an array or linked list is one way to implement it.

3. Arrays and basic array operations

Definition and properties

An array stores a finite collection of elements of the same type under one name. Each element is accessed by an index/subscript. In a conventional array implementation, elements occupy consecutive memory locations.

```
A = [10, 20, 30, 40, 50]
Index: 0   1   2   3   4
A[2] = 30
```

Traversal

```
TRAVERSE(A, n)
    for i ← 0 to n - 1
        process A[i]
```

Time: O(n), because each of the n elements is visited once. Auxiliary space: O(1) for an iterative traversal.

Linear search

```
LINEAR_SEARCH(A, n, key)
    for i ← 0 to n - 1
        if A[i] = key
            return i
    return NOT_FOUND
```

Case	Time
Best case — first element matches	O(1)
Average case	O(n)
Worst case — last element or absent	O(n)

Insertion at position k

To insert in the middle, shift elements from the end toward the right to make room. Validate that the array has spare capacity and that the requested position is valid.

```
INSERT(A, n, k, item)       // k is a 0-based position
    if n = capacity
        report OVERFLOW
    for j ← n - 1 down to k
        A[j + 1] ← A[j]
    A[k] ← item
    n ← n + 1
```

Example

Start: [10, 20, 30, 40, 50]. Insert 15 at position 1 (0-based). Shift 50, 40, 30, and 20 one place right; then write 15 at index 1. Result: [10, 15, 20, 30, 40, 50].

Deletion at position k

```
DELETE(A, n, k)
    if n = 0
        report UNDERFLOW
    item ← A[k]
    for j ← k to n - 2
        A[j] ← A[j + 1]
    n ← n - 1
    return item
```

After deletion, elements to the right of the deleted item shift one place left. Insertion and deletion at the beginning/middle are O(n) in the worst case; deletion at the end can be O(1) if only the logical size is updated.

Array operation complexity

Operation	Typical complexity	Reason
Access A[i]	O(1)	Index gives direct location.
Traversal	O(n)	Visit all elements.
Linear search	O(n)	May inspect all elements.
Insert at beginning/middle	O(n)	Shift elements.
Delete at beginning/middle	O(n)	Shift elements left.

4. Linked lists

Core idea

A linked list consists of nodes. A node stores data and one or more links/pointers to other nodes. Unlike an array, nodes do not need to occupy consecutive memory locations.

```
Singly linked list:
HEAD → [data | next] → [data | next] → [data | NULL]

Doubly linked list:
NULL ← [prev | data | next] ■ [prev | data | next] → NULL
```

Types

Type	Node links / structure	Key point
Singly linked list	Each node points to the next node.	Forward traversal only; final next = NULL.
Circular singly linked list	Last node points back to the first node.	No NULL at the end of the cycle; stop after a full cycle.
Doubly linked list	Each node has prev and next links.	Forward and backward traversal; more pointer memory.

Basic node model

```
Node:
    data
    next

head = pointer to first node
empty list: head = NULL
```

Traversal

```
TRAVERSE(head)
    current ← head
    while current ≠ NULL
        visit current.data
        current ← current.next
```

Insertion at beginning

```
INSERT_FRONT(head, item)
    new ← create node
    new.data ← item
    new.next ← head
    head ← new
    return head
```

Deletion at beginning

```
DELETE_FRONT(head)
    if head = NULL
        report UNDERFLOW
    temp ← head
    head ← head.next
    free temp
    return head
```

Searching

```
SEARCH(head, key)
    current ← head
    while current ≠ NULL
        if current.data = key
            return current
        current ← current.next
    return NOT_FOUND
```

Complexity and comparison

Operation	Singly linked list (typical)
Access/search by value	O(n)
Traversal	O(n)
Insert at head	O(1)
Delete at head	O(1)
Insert after a known node	O(1)
Insert at a specified index	O(n) to find position

Pointer update matters: before deleting a node, keep track of the previous node and reconnect its next link to the node after the one being removed. Handle an empty list and deletion of the head separately.

Applications of lists

- Maintaining dynamic collections where size changes frequently.
- Representing polynomials as terms (coefficient, exponent) and combining like exponents.
- Implementing other structures, including stacks and queues.

5. Stack: operations, representations and applications

Definition and principle

A stack is a linear structure in which insertion and deletion occur at one end called the **TOP**. It follows **LIFO — Last In, First Out**. Think of a stack of plates: the most recently placed plate is removed first.

```
TOP
 ↓
[30]  ← most recently pushed
[20]
[10]
```

Operations

Operation	Meaning	Error condition
push(x)	Insert x at TOP.	Overflow if a fixed-size stack is full.
pop()	Remove and return TOP.	Underflow if stack is empty.
peek()/top()	Read TOP without removing it.	Empty stack.
isEmpty()	Check whether stack has no elements.	—
isFull()	Check capacity in array implementation.	—

Array representation

For a stack array, TOP is often initialized to -1. The stack is empty when TOP = -1; for capacity MAX, it is full when TOP = MAX - 1.

```
PUSH(S, item)
    if TOP = MAX - 1
        report OVERFLOW
    TOP ← TOP + 1
    S[TOP] ← item

POP(S)
    if TOP = -1
        report UNDERFLOW
    item ← S[TOP]
    TOP ← TOP - 1
    return item
```

Linked representation

A linked stack uses nodes and a pointer TOP to the first node. Push inserts a node at the head; pop removes the head. These operations are O(1) when performed correctly. A linked stack grows dynamically until memory is unavailable.

Applications of stack

- Function calls and recursion: the call stack stores return information and local context.
- Expression conversion and evaluation (infix, postfix, prefix).
- Parentheses/bracket matching and syntax parsing.
- Undo/redo histories and backtracking.
- Reversing a sequence: push characters, then pop them in reverse order.

Operation	Array stack (typical)
push	O(1)

Operation	Array stack (typical)
pop	O(1)
peek	O(1)
Search for arbitrary item	O(n)

6. Expressions: infix, postfix and evaluation

Expression forms

Form	Operator position	Example
Infix	Between operands	A + B
Prefix	Before operands	+ A B
Postfix	After operands	A B +

Postfix expressions do not require parentheses to indicate operator precedence. A stack is commonly used to convert infix expressions and evaluate postfix expressions.

Infix-to-postfix: precedence

Operator(s)	Typical precedence	Associativity
( )	Grouping	—
^ / exponent	Highest among common arithmetic operators	Often right-to-left
* / %	Next	Left-to-right
+ -	Lower	Left-to-right

Conversion method (operator-stack approach)

- Scan the infix expression from left to right.
- If the token is an operand, append it to the postfix output.
- If it is an opening parenthesis, push it.
- If it is an operator, pop operators of higher precedence (and equal precedence where associativity requires it), then push the current operator.
- If it is a closing parenthesis, pop to the matching opening parenthesis and discard the parentheses.
- At the end, pop the remaining operators.

Worked example

Convert **A + B * C**.

Token	Action	Postfix so far
A	Output operand A	A
+	Push +	A
B	Output operand B	AB
*	Higher precedence than +, push *	AB
C	Output operand C	ABC
End	Pop * then +	ABC*+

Result: A + B * C → ABC*+. Multiplication is performed before addition.

Postfix evaluation

- Scan left to right. Push each operand onto the stack.
- When an operator appears, pop the required operands, apply the operator, and push the result.
- For a binary operator, the first pop is the right operand and the second pop is the left operand.

Example: 6 2 3 + -

Token	Action	Stack after action (bottom → top)
6	Push 6	[6]
2	Push 2	[6, 2]
3	Push 3	[6, 2, 3]
+	3 + 2 = 5	[6, 5]
-	6 - 5 = 1	[1]

Common mistake: subtraction and division are not commutative. Pop right operand first, then left operand; calculate left operator right.

7. Queue: operations, representations and queue types

Definition and principle

A queue is a linear structure in which insertion takes place at the **REAR** and deletion at the **FRONT**. It follows **FIFO — First In, First Out**, like a line of people waiting.

```
FRONT → [10] [20] [30] ← REAR
dequeue removes 10; enqueue adds at REAR
```

Core operations

Operation	Meaning
enqueue(x)	Insert x at the rear.
dequeue()	Remove and return the front element.
front()/peek()	Read the front element without removing it.
isEmpty()	Check whether no element exists.
isFull()	Check whether a fixed-capacity array queue is full.

Array representation and common issue

A simple linear array queue tracks FRONT and REAR. Repeated deletions can leave unused slots at the beginning; the rear may reach the array end even when earlier slots are free. A circular queue solves this by wrapping indices around.

```
ENQUEUE(Q, item)  // simple conceptual version
    if queue is full
        report OVERFLOW
    if queue is empty
        FRONT ← 0
    REAR ← REAR + 1
    Q[REAR] ← item

DEQUEUE(Q)
    if queue is empty
        report UNDERFLOW
    item ← Q[FRONT]
    FRONT ← FRONT + 1
    return item
```

Circular queue

A circular queue treats the array as a ring. Indices wrap using modulo arithmetic. For an implementation that reserves one slot to distinguish full from empty, a common full test is (REAR + 1) mod capacity = FRONT; exact tests depend on the chosen convention.

```
next_index = (index + 1) % capacity
```

Queue types

Type	Idea / distinguishing feature
Linear queue	Insertion at rear, deletion at front.
Circular queue	Last array position connects logically to first; reuses free slots.
Deque	Insertion and deletion are permitted at both ends.
Priority queue	Removal is based on priority, not necessarily arrival order.

Array vs linked queue

Feature	Array-based	Linked implementation
Storage	Fixed capacity in a basic array.	Dynamic nodes.
Pointers	Uses indices FRONT/REAR.	Uses front/rear node pointers.
Overflow	When capacity is reached.	When memory allocation fails.
Enqueue/dequeue	Usually O(1) with circular array and careful index handling.	O(1) when front and rear pointers are maintained.

Applications

- Scheduling and service systems, printer queues, task queues.
- Breadth-first processing and buffering data streams.
- Handling requests in arrival order.

8. Algorithms: definition, properties and specification

Definition

An algorithm is a finite sequence of clear, unambiguous instructions that transforms legitimate input into the required output.

Characteristics

Characteristic	Meaning
Input	Zero or more externally supplied quantities.
Output	At least one result is produced.
Definiteness	Every instruction is clear and unambiguous.
Finiteness	Terminates after a finite number of steps for valid cases.
Effectiveness / efficiency	Instructions are basic enough to be carried out; the algorithm should use resources sensibly.

Ways to specify an algorithm

Method	Description	Limitation / benefit
Natural language	Steps in ordinary language.	Easy to read, but can be ambiguous.
Pseudocode	Structured, language-independent instructions.	Clear and close to implementation.
Flowchart	Connected shapes represent steps and control flow.	Visual, but can become cumbersome for large algorithms.

Pseudocode conventions

- Use a heading such as ALGORITHM Name(parameters).
- Document problem description, input and output.
- Use assignment arrow ←, conditions, loops and return statements.
- Indent blocks consistently; use comments such as // explanation.

Example: Euclid's GCD algorithm

The uploaded algorithm notes explain gcd(m, n) through repeated remainders: gcd(m, n) = gcd(n, m mod n). When n becomes 0, m is the answer.

```
EUCLID_GCD(m, n)
    while n ≠ 0
        r ← m mod n
        m ← n
        n ← r
    return m
```

Dry run: gcd(60, 24)

m	n	r = m mod n
60	24	12
24	12	0

When n = 0, return m = 12. Therefore gcd(60, 24) = 12.

9. Algorithmic problem-solving process

Step	What to do
1. Understand the problem	Read carefully; identify inputs, outputs, constraints, and edge cases.
2. Make key decisions	Consider computational resources; choose exact or approximate solving where appropriate; select a design strategy.
3. Design the algorithm	Choose a suitable method and data structure.
4. Prove correctness	Show the algorithm gives the required result for every valid input.
5. Analyze efficiency	Estimate time and extra-space growth as input size increases.
6. Implement and test	Translate into code and test normal, boundary, and invalid cases.

Computational models and solution styles

- **Sequential algorithm:** instructions are executed one after another.
- **Parallel algorithm:** uses concurrent operations where supported.
- **Exact algorithm:** returns an exact correct solution.
- **Approximation algorithm:** returns a useful approximate solution when exact solving is impractical.

Common design strategies named in the source

Strategy	High-level idea
Brute force	Try a direct, straightforward method; may examine many candidates.
Divide and conquer	Break a problem into smaller subproblems, solve them, then combine.
Dynamic programming	Store/reuse results of overlapping subproblems.
Greedy technique	Make a locally attractive choice at each step.

Common problem types in algorithm analysis

- Sorting: arrange items in an order.
- Searching: locate an item matching a key.
- String processing: work with sequences of characters.
- Graph problems: represent and solve relationship/network problems.
- Combinatorial problems: select, arrange, or evaluate combinations.
- Geometric and numerical problems: process geometric objects or numerical values.

10. Time and space complexity

Why analyze algorithms?

Algorithm analysis estimates how resource use grows with input size. It helps compare algorithms independently of a particular computer or programming language.

Time complexity

Time complexity describes the growth in the number of basic operations as a function of input size n. It is not simply clock time in seconds.

Space complexity

Space complexity describes memory requirements as input size grows. It can include instruction/code space, data space, and environment/call-stack space. **Auxiliary space** is the extra working memory used beyond the input representation, depending on the convention used in a question.

Common growth rates

Growth	Name	Typical intuition
O(1)	Constant	A fixed number of steps.
O(log n)	Logarithmic	Problem size shrinks by a constant factor per step.
O(n)	Linear	One pass over n items.
O(n log n)	Linearithmic	Efficient comparison-sorting growth pattern.
O(n²)	Quadratic	Two nested passes over n items.
O(2■)	Exponential	Work doubles with each additional input unit.
O(n!)	Factorial	Enumerating all permutations.

Analysis framework

- Identify the input-size parameter (often n).
- Choose the basic operation that dominates the running time.
- Count how many times the basic operation executes.
- Express the count as a function of n and simplify asymptotically.
- For recursive algorithms, form and solve a recurrence when appropriate.

Best, average and worst case

Case	Meaning
Best case	Minimum work for inputs of size n.
Average case	Expected work under a stated input distribution.
Worst case	Maximum work over inputs of size n.

For linear search: best case O(1), worst case O(n). Always state the case when the question asks for it.

11. Asymptotic notation: O, Ω and Θ

Asymptotic notation describes how a function grows for sufficiently large n, ignoring constant factors and lower-order terms.

Big-O: upper bound

f(n) ∈ O(g(n)) means that, beyond some threshold, f(n) is bounded above by a constant multiple of g(n). It is commonly used to express an upper growth bound.

Big-Omega: lower bound

f(n) ∈ Ω(g(n)) means that, beyond some threshold, f(n) is bounded below by a positive constant multiple of g(n).

Big-Theta: tight bound

f(n) ∈ Θ(g(n)) means both O(g(n)) and Ω(g(n)) hold: the two functions have the same asymptotic order of growth.

Notation	Memory cue	Example
O(g(n))	At most / upper bound	$3n^2 + 5n + 2 \in O(n^2)$
Ω(g(n))	At least / lower bound	$3n^2 + 5n + 2 \in \Omega(n^2)$
Θ(g(n))	Same order / tight bound	$3n^2 + 5n + 2 \in \Theta(n^2)$

Simplifying growth expressions

- Drop constant factors: 7n becomes Θ(n).
- Keep the dominant term: $n^2 + 5n + 2$ becomes $\Theta(n^2)$.
- For a sum of positive growth terms, the dominant term determines the asymptotic order.

Quick examples

Expression	Tight order
12	Θ(1)
4n + 9	Θ(n)
$n^2 + 3n + 8$	$\Theta(n^2)$
5n log n + 2n	Θ(n log n)
2■ + n^3	Θ(2■)

Common mistake: Big-O is not automatically “worst case,” and Big-Ω is not automatically “best case.” The notation describes a function bound; best/worst case describe which inputs are being analyzed.

12. Mathematical analysis: non-recursive algorithms

Example A: maximum element

```
MAX_ELEMENT(A, n)
    max ← A[0]
    for i ← 1 to n - 1
        if A[i] > max
            max ← A[i]
    return max
```

The loop compares each remaining element with the current maximum, giving n – 1 comparisons. Time is $\Theta(n)$; the iterative algorithm uses $\Theta(1)$ auxiliary space.

Example B: nested loops

```
for i ← 0 to n - 1
    for j ← 0 to n - 1
        perform constant-time operation
```

The inner operation executes $n \times n = n^2$ times, so the time complexity is $\Theta(n^2)$.

Example C: matrix multiplication (basic triple-loop method)

```
for i ← 0 to n - 1
    for j ← 0 to n - 1
        C[i][j] ← 0
        for k ← 0 to n - 1
            C[i][j] ← C[i][j] + A[i][k] * B[k][j]
```

The three loops each run approximately n times; the basic method performs $\Theta(n^3)$ arithmetic operations.

Example D: binary digit count by repeated division

```
BINARY_DIGITS(n)
    count ← 1
    while n > 1
        n ← floor(n / 2)
        count ← count + 1
    return count
```

Each iteration reduces n by about a factor of 2, so the number of iterations is $\Theta(\log n)$ for positive n under this formulation.

Loop-counting rules

- A single loop from 1 to n with constant work per iteration $\rightarrow \Theta(n)$.
- Two nested n-sized loops $\rightarrow \Theta(n^2)$.
- Consecutive loops add their costs: $\Theta(n) + \Theta(n^2) = \Theta(n^2)$.
- A loop that halves/doubles its controlling value each time often gives $\Theta(\log n)$.

13. Mathematical analysis: recursive algorithms

What is recursion?

A recursive algorithm calls itself on a smaller instance. A correct recursive design needs a base case and progress toward that base case.

Factorial

```
FACTORIAL(n)
    if n = 0
        return 1
    return n * FACTORIAL(n - 1)
```

Recurrence: T(n) = T(n – 1) + Θ(1), with T(0) = Θ(1). There are n recursive levels, so time is Θ(n); the call stack is Θ(n).

Tower of Hanoi

To move n disks from source A to destination C using auxiliary peg B: move n–1 disks from A to B, move the largest disk from A to C, then move n–1 disks from B to C.

```
TOH(n, A, C, B)
    if n = 1
        move disk A → C
    else
        TOH(n - 1, A, B, C)
        move disk A → C
        TOH(n - 1, B, C, A)
```

Recurrence: T(n) = 2T(n – 1) + 1, with T(1) = 1. Solving gives T(n) = 2■ – 1 moves, so time is Θ(2■).

Recursive binary-digit / binary-recursive examples

When a recursive call reduces the input to roughly half, the recurrence often includes T(n/2). For example, T(n) = T(n/2) + Θ(1) solves to Θ(log n). The exact recurrence must match the actual algorithm.

Recursion analysis recipe

- Write the base case and the cost outside recursive calls.
- Count how many recursive calls are made.
- Write a recurrence such as T(n) = T(n – 1) + c or T(n) = 2T(n – 1) + c.
- Solve by expansion/substitution, a recursion tree, or a suitable recurrence method.

Factorial and Tower of Hanoi are different: factorial has one recursive call per level (Θ(n)); Hanoi has two recursive calls per level (Θ(2■)).

14. One-page DSA + algorithm complexity cheat sheet

Structure / operation	Typical time	Remember
Array index access	O(1)	Direct index.
Array traversal / linear search	O(n)	May inspect every element.
Array insert/delete at middle	O(n)	Shifting is required.
Singly linked list traversal/search	O(n)	Move node by node.
Linked list insert at head	O(1)	Update head and one link.
Stack push/pop/peek	O(1)	Operate at TOP.
Queue enqueue/dequeue	O(1)	With correct circular-array or linked implementation.
Iterative maximum element	Θ(n)	One pass.
Two nested n loops	$\Theta(n^2)$	n × n operations.
Basic triple-loop matrix multiply	$\Theta(n^3)$	Three n-sized loops.
Factorial recursion	Θ(n) time, Θ(n) stack	One recursive call per level.
Tower of Hanoi	Θ(2■) time	T(n)=2T(n–1)+1.

Fast memory triggers

If question says…	Think…
Last in, first out	Stack / TOP
First in, first out	Queue / FRONT and REAR
Contiguous memory and index	Array
Nodes and links	Linked list
Clear finite instructions	Algorithm
Upper / lower / tight bound	O / Ω / Θ
Repeated halving	Logarithmic growth
Nested loops	Multiply iteration counts
Self-calling procedure	Recursion + base case + recurrence

15. Exam-ready long-answer frameworks

Question type A: Explain a data structure

Suggested structure: (1) Definition, (2) principle/diagram, (3) representation, (4) operations with pseudocode, (5) complexity, (6) applications, (7) advantages/limitations.

Question type B: Explain stack and its applications

Define stack and LIFO; draw TOP; explain push, pop, peek and overflow/underflow; show array or linked representation; explain expression conversion/evaluation, function calls and reversing; conclude with O(1) push/pop.

Question type C: Compare array and linked list

Compare memory layout, access, insertion/deletion, size flexibility, extra memory and use cases. State that arrays support direct indexed access while linked lists require traversal.

Question type D: Explain asymptotic notation

Define O, Ω and Θ; give an intuitive upper/lower/tight-bound interpretation; use one polynomial example; explain dominant terms and why constants are ignored.

Question type E: Analyze an algorithm

State input size n; identify the basic operation; count its executions; write a sum or recurrence; simplify to asymptotic order; mention auxiliary space if asked.

Question type F: Queue and its types

Define FIFO; draw FRONT and REAR; explain enqueue/dequeue and empty/full cases; discuss linear-array limitations; describe circular queue, deque and priority queue; include applications.

> **For a 10-mark answer, diagrams and a dry run often make the explanation easier to follow. Keep the pseudocode aligned with the explanation.**

16. Practice questions with answer cues

Short-answer practice

1. Define a data structure and list four basic operations.

Answer cue: Organize/store/retrieve data; traversal, searching, insertion, deletion (sorting/merging also common).

2. What is an ADT?

Answer cue: Logical model of data and allowed operations, independent of implementation.

3. Differentiate stack and queue.

Answer cue: Stack: LIFO, insertion/deletion at TOP. Queue: FIFO, insert at REAR and delete at FRONT.

4. Why can insertion into an array be O(n)?

Answer cue: Elements may need shifting to create a free position.

5. State three linked-list types.

Answer cue: Singly, circular singly, doubly linked list.

6. What are overflow and underflow?

Answer cue: Overflow: insertion attempted when fixed capacity is full. Underflow: deletion attempted when structure is empty.

7. Define O, Ω and Θ.

Answer cue: Upper, lower and tight asymptotic bounds.

8. What is the base case in recursion?

Answer cue: The stopping condition solved without further recursive calls.

Trace / problem practice

Question	Answer cue
Find gcd(48, 18) using Euclid's algorithm.	48 mod 18 = 12; 18 mod 12 = 6; 12 mod 6 = 0 → gcd = 6.
Insert 25 at index 2 in [10, 20, 30, 40].	Shift 40 and 30 right; result [10, 20, 25, 30, 40].
Convert A * (B + C) to postfix.	ABC+*.
Evaluate postfix 5 2 3 * +.	2×3=6; 5+6=11.
Analyze two nested loops each running n times.	Θ(n²).
Give the Tower of Hanoi recurrence.	T(n)=2T(n–1)+1, T(1)=1; moves = 2■–1.

Self-test: mark each true or false

- 1. A stack follows FIFO.
- 2. A queue inserts at the rear and deletes from the front.
- 3. Linked-list nodes must always be contiguous in memory.
- 4. Array access by a valid index is typically O(1).
- 5. Θ(g(n)) implies both O(g(n)) and Ω(g(n)).
- 6. The basic triple-loop matrix multiplication algorithm is Θ(n³).
- 7. An empty stack may cause underflow on pop.
- 8. Tower of Hanoi requires 2■–1 moves for n disks.

Answer key: 1 False • 2 True • 3 False • 4 True • 5 True • 6 True • 7 True • 8 True

Final 15-minute revision

Topic	Must remember
Data structure	Organizes data for efficient processing.
ADT	What operations do; not how implemented.
Array	Contiguous storage; indexed access O(1); middle insert/delete O(n).
Linked list	Nodes and links; traversal/search O(n); head insertion O(1).
Stack	LIFO; TOP; push/pop; overflow/underflow; expression operations.
Queue	FIFO; FRONT/REAR; enqueue/dequeue; circular queue reuses slots.
Algorithm	Clear, finite, unambiguous steps; input and output.
Analysis	Count basic operations as a function of input size.
Asymptotic notation	O upper; Ω lower; Θ tight.
Recursion	Base case + progress + recurrence.
Canonical complexities	Linear search O(n); nested loops Θ(n²); factorial Θ(n); Hanoi Θ(2■).

End of combined notes. Revise the definitions, trace the examples by hand, and practise writing algorithms without looking at the solution.